


# University of Chile

## sin globito no hay fiesta

Diego Arias, Gabriel Carmona, Martín Ruiz-Tagle

2026-03-04

Contest 2
Data Structures 2
Number Theory 6
Numerical 8
Geometry 11
3D Geometry 15
Strings 15
Graphs 17
Flow 22
Various 24

# Contest

## Template

```cpp
#include <bits/stdc++.h>
using namespace std;

template <class T>
using vec = vector<T>;
using ll = int64_t;
using pii = pair<int, int>;
using vi = vec<int>;
```
4d1a3a,**4d1a3a**

```cpp
#define pb push_back
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = a; i < (b); ++i)

// cin.tie(0)->sync_with_stdio(0);
// cin.exceptions(cin.failbit);
```
2a6b51,**fd8a65**

## Bash

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++20 -
fsanitize=undefined,address'
```
503d21,**503d21**

## Vim

```
set cin aw ai is ts=4 sw=4 nu noeb bg=dark ru cb=unnamedplus
sy on
ino {<CR> {<CR>}<Esc>ko
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
md5sum \| cut -c-6
```
3e241c,**3e241c**

## troubleshoot.txt

**Pre-submit:**
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.
**Wrong answer:**
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing $N$ and $M$, $i$ and $j$, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the `STL` functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet 🧑🚽.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
**Runtime error:**
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).
**Time limit exceeded:**
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?
**Memory limit exceeded:**
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

## Binary Search

**description:** Returns first $x \in [\text{lo}, \text{hi})$ that is false. `lo` and `hi` must be of the same type or there is weird behaviour.

```cpp
ll res = *ranges::partition_point(views::iota(lo, hi),
  [&](ll x) -> bool { return x < 0; });
```
6ff8a4,**6ff8a4**

# Data Structures

## Faster Hashmap

**description:** Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

```cpp
#include <bits/extc++.h>
using namespace __gnu_pbds;

struct chash {
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x *
C); }
};
gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1 << 16});
```
a8ddf0,**a8ddf0**

## Order Statistic Tree

**time:** $O(\log n)$

```cpp
#include <bits/extc++.h>
using namespace __gnu_pbds;

template <class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
                  tree_order_statistics_node_update>;
```
cd2981,**cd2981**

## Union Find with Rollback

**description:** If undo is not needed remove `st`, `time()`, `rollback()` and add path compression if necessary.
**time:** $O(\log n)$

```cpp
struct RollbackUF {
  vi e; vec<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
```
f508ab,**f508ab**

```cpp
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
```
7fbe65,**39a27a**

```cpp
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```
b34fa4,**58e065**

## Convex Hull Trick

**description:** Add lines of the form $kx + m$, and query the maximum value at some point $x$.
**time:** $O(\log n)$

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line &o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
```
a3ffb4,**a3ffb4**
```
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end())
      return x->p = inf, 0;
    if (x->k == y->k)
      x->p = x->m > y->m ? inf : -inf;
```
92ccfc,**1202d3**
```
    else
      x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z))
      z = erase(z);
```
e84ba2,**e189b8**
```
    if (x != begin() && isect(--x, y))
      isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    auto l = *lower_bound(x);
    return l.k * x + l.m;
```
752091,**fa8887**
```
  }
};
```
e0360a,**46d58a**

## Coordinate Compression

**usage:** `vec<reference_wrapper<int>> X; X.pb(ref(x)); auto values = compress_coords(X);`

```
auto compress_coords(auto &X) {
  vi values;
  values.reserve(sz(X));
  sort(all(X));
  int i = -1, prev;
  for (auto &x : X) {
    if (i == -1 || x != prev) {
      i++;
```
d87abb,**d87abb**
```
      prev = x;
      values.pb(x);
    }
    x.get() = i;
  }
  return values;
}
```
e65ee5,**c038b3**

## Sparse Table

**time:** $O(n \log n)$ preprocessing and $O(1)$ queries

```
template <class T> struct SparseTable {
  vec<vec<T>> jmp;
  SparseTable(vec<T> &a) : jmp(1, vec<T>(sz(a))) {
    rep(i,0,sz(a))
      jmp[0][i] = a[i];
    for (int pw = 1, k = 1; pw * 2 <= sz(a); pw *= 2, ++k) {
      jmp.emplace_back(sz(a) - pw * 2 + 1);
      rep(j,0,sz(jmp[k]))
```
0f17ae,**0f17ae**
```
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int b, int e) { // [b, e)
    int dep = 31 - __builtin_clz(e - b);
    return min(jmp[dep][b], jmp[dep][e - (1 << dep)]);
  }
};
```
338c4e,**1d8642**

## Iterative Segment Tree

**description:** Iterative segment tree, with point update and range queries. You can replace $s$ with a fast hashmap to get a dynamic segment tree.
**time:** $O(\log n)$

```
template <class T, auto op, T e> struct SegmentTree {
  int n; vec<T> s;
  SegmentTree(int n) : n(n), s(2*n, e) {}
  T query(int a, int b) { // [a, b)
    T ra = e, rb = e;
    for (a += n, b += n; a < b; a /= 2, b /= 2) {
      if (a % 2) ra = op(ra, s[a++]);
      if (b % 2) rb = op(s[--b], rb);
```
5c1052,**5c1052**
```
    }
    return op(ra, rb);
  }
  void upd(int i, T u) {
    for (i += n, s[i] = u; i /= 2;)
      s[i] = op(s[i*2], s[i*2+1]);
  }
  void build(vec<T> &a) {
```
8e713a,**8b9126**
```
    rep(i,0,n) s[i + n] = a[i];
    for (int i = n - 1; i >= 0; i--)
      s[i] = op(s[i*2], s[i*2+1]);
  }
};
```
0f5461,**647b31**

## Fenwick Tree

**description:** Fenwick tree, with point addition and sum range queries.
**time:** $O(\log n)$

```
struct FT {
  vec<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // [0, pos)
    ll res = 0;
```
8155dd,**8155dd**
```
    for (; pos > 0; pos &= pos - 1) res += s[pos - 1];
    return res;
  }
  int lower_bound(ll sum) { // s[0, pos] >= sum
    // returns n if no pos satisfies >= sum.
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
```
834eb9,**781a10**
```
        pos += pw, sum -= s[pos - 1];
    }
    return pos;
  }
};
```
f823dd,**84b367**

## Compressed Fenwick Tree 2D

**description:** 2D fenwick tree with compressed $y$ axis, with point addition and sum range queries. Call `fakeUpdate` on the positions that will receive updates before `init`.
**time:** $O(\log^2 n)$

```
#include "fenwick_tree.h"
struct FT2 {
  vec<vi> ys; vec<FT> ft;
  FT2(int lim_x) : ys(lim_x) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  }
  void init() {
```
17d496,**17d496**
```
    for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
  }
  int ind(int x, int y) {
    return int(lower_bound(all(ys[x]), y) - ys[x].begin());
  }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
```
fcb320,**ea1c33**
```
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```
22070e,**84c281**

## Split Function Segment Tree

```
int split(int tl, int tr) {
  int pw2 = 1 << __lg(tr - tl);
```

```
  return min(tl + pw2, tr - pw2 / 2);
}
```
cc4bad,**cc4bad**

## Segment Tree

**description:** Recursive segment tree, with point update and range queries.
**time:** $O(\log n)$

```
template <class T, auto op, T e> struct SegmentTree {
  int n; vec<T> s;
  SegmentTree(int n) : n(n), s(2*n, e) {}
  T query(int l, int r) { return query(l, r, 1, 0, n); }
  T query(int l, int r, int v, int tl, int tr) {
    if (r <= tl || tr <= l) return e;
    if (l <= tl && tr <= r) return s[v];
    int tm = split(tl, tr);
```
6acfb8,**6acfb8**
```
    return op(query(l, r, 2*v, tl, tm),
              query(l, r, 2*v+1, tm, tr));
  }
  void upd(int pos, T u) { return upd(pos, u, 1, 0, n); };
  void upd(int pos, T u, int v, int tl, int tr) {
    if (tr - tl == 1) {
      s[v] = u;
    } else {
```
507b34,**872b56**
```
      int tm = split(tl, tr);
      if (pos < tm) upd(pos, u, 2*v, tl, tm);
      else upd(pos, u, 2*v+1, tm, tr);
      s[v] = op(s[2*v], s[2*v+1]);
    }
  }
  void build(vec<T>& a) {
    int pw2 = bit_ceil(size(a));
```
b86039,**7ec70a**
```
    for (int i = 0; i < n; i++)
      s[(i + pw2) % n + n] = a[i];
    for (int i = n - 1; i >= 1; i--)
      s[i] = op(s[2*i], s[2*i+1]);
  }
};
```
aaed7b,**6f5920**

## Persistent Segment Tree

**time:** $O(\log n)$

```
template <class T, auto op, T e> struct SegmentTree {
  struct Node {
    int l = 0, r = 0;
    T x = e;
  };
  int n;
  vec<Node> s;
  SegmentTree(int n) : n(n), s(1) {}
```
3c60aa,**3c60aa**
```
  int copy(int v) {
    s.emplace_back(s[v]);
    return sz(s) - 1;
  };
  T query(int l, int r, int v) { return query(l, r, v, 0,
n); }
  T query(int l, int r, int v, int tl, int tr) { // [l, r)
    if (r <= tl || tr <= l) return e;
    if (l <= tl && tr <= r) return s[v].x;
```
03de0b,**be2788**
```
    int tm = (tl+tr)/2;
    return op(query(l, r, s[v].l, tl, tm),
              query(l, r, s[v].r, tm, tr));
  }
  int upd(int pos, T u, int v) { return upd(pos, u, v, 0,
n); }
  int upd(int pos, T u, int v, int tl, int tr) {
    v = copy(v);
    if (tr - tl == 1) {
```
bfdd47,**730a20**
```
      s[v].x = u;
    } else {
      int tm = (tl+tr)/2;
      if (pos < tm) s[v].l = upd(pos, u, s[v].l, tl, tm);
      else s[v].r = upd(pos, u, s[v].r, tm, tr);
      s[v].x = op(s[s[v].l].x, s[s[v].r].x);
    }
    return v;
```
ff767d,**7a3991**
```
  }
};
```
e0360a,**6f4c68**

## Tag Type

**description:** Tag operator type used for lazy propagation in lazy structures.

```
struct Tag {
  ll b = 1, c = 0;
  ll map(ll x, int l, int r) const {
    return (x * b % mod + c * (r - l) % mod) % mod;
  }
  Tag operator()(Tag nu) const {
    return {
      b * nu.b % mod,
```
c61f8f,**c61f8f**
```
      (c * nu.b + nu.c) % mod
    };
  }
};
```
e4c61d,**4c95b9**

## Lazy Segment Tree

**time:** $O(\log n)$

```
template <class T, auto op, T e,
          class U, U id> struct SegmentTree {
  int n; vec<T> s; vec<U> lz;
  SegmentTree(int n) : n(n), s(2*n, e), lz(n, id) {}
  T query(int l, int r) { return query(l, r, 1, 0, n); }
  T query(int l, int r, int v, int tl, int tr) {
    if (r <= tl || tr <= l) return e;
    if (l <= tl && tr <= r) return s[v];
```
1ff1d1,**1ff1d1**
```
    push(v, tl, tr);
    int tm = split(tl, tr);
    return op(query(l, r, 2*v, tl, tm),
              query(l, r, 2*v+1, tm, tr));
  }
  void upd(int l, int r, U u) { return upd(l, r, u, 1, 0,
n); }
  void upd(int l, int r, U u, int v, int tl, int tr) {
    if (r <= tl || tr <= l) return;
```
58741e,**052705**
```
    if (l <= tl && tr <= r) {
      s[v] = u.map(s[v], tl, tr);
      if (v < n) lz[v] = lz[v](u);
      return;
    }
    push(v, tl, tr);
    int tm = split(tl, tr);
    upd(l, r, u, 2*v, tl, tm), upd(l, r, u, 2*v+1, tm, tr);
```
ee3a36,**44a818**
```
    s[v] = op(s[2*v], s[2*v+1]);
  }
  void push(int v, int tl, int tr) {
    int tm = split(tl, tr);
    upd(tl, tm, lz[v], 2*v, tl, tm);
    upd(tm, tr, lz[v], 2*v+1, tm, tr);
    lz[v] = id;
  }
```
a2cd0c,**686671**
```
  void build(vec<T>& a) {
    auto pw2 = bit_ceil(size(a));
    for (int i = 0; i < n; i++)
      s[(i + pw2) % n + n] = a[i];
    for (int i = n - 1; i >= 1; i--)
      s[i] = op(s[2*i], s[2*i+1]);
  }
};
```
5c6916,**891cf1**

## Persistent Lazy Segment Tree

**time:** $O(\log n)$

```
template <class T, auto op, T e,
          class U, U id> struct SegmentTree {
  struct Node {
    int l = 0, r = 0;
    T x = e;
    U lz = id;
  };
  int n;
```
660272,**660272**
```
  vec<Node> s;
  SegmentTree(int n) : n(n), s(1) {}
  int copy(int v) {
    s.emplace_back(s[v]);
    return sz(s) - 1;
  };
  T query(int l, int r, int v) { return query(l, r, v, 0,
n); }
  T query(int l, int r, int v, int tl, int tr) { // [l,r)
    if (r <= tl || tr <= l) return e;
    if (l <= tl && tr <= r) return s[v].x;
    int t = sz(s);
    v = push(v, tl, tr);
    int tm = (tl+tr)/2;
    T val = op(query(l, r, s[v].l, tl, tm),
               query(l, r, s[v].r, tm, tr));
    s.resize(t);
```
170e28,**c5fb2d**
42b245,**2289c5**
```
    return val;
  }
  int upd(int l, int r, U u, int v) { return upd(l, r, u, v,
0, n); }
```

```
  int upd(int l, int r, U u, int v, int tl, int tr) {
    if (r <= tl || tr <= l) return v;
    if (l <= tl && tr <= r) {
      v = copy(v);
      s[v].x = u.map(s[v].x, tl, tr);
```
08ffe8,**a48abb**
```
      s[v].lz = s[v].lz(u);
    } else {
      v = push(v, tl, tr);
      int tm = (tl+tr)/2;
      s[v].l = upd(l, r, u, s[v].l, tl, tm);
      s[v].r = upd(l, r, u, s[v].r, tm, tr);
      s[v].x = op(s[s[v].l].x, s[s[v].r].x);
    }
```
9b00c1,**d3fba1**
```
    return v;
  }
  int push(int v, int tl, int tr) {
    v = copy(v);
    if (tr - tl == 1) return v;
    int tm = (tl+tr)/2;
    s[v].l = upd(tl, tm, s[v].lz, s[v].l, tl, tm);
    s[v].r = upd(tm, tr, s[v].lz, s[v].r, tm, tr);
```
382a1b,**497323**
```
    s[v].lz = id;
    return v;
  }
};
```
3270e1,**d063bf**

## Treap Node

```
mt19937 rng(chrono::high_resolution_clock::now()
          .time_since_epoch().count());
struct Node {
  Node *l = 0, *r = 0;
  int y, c = 1;
  ll x;
  Node(ll x) : y(int(rng())), x(x) {}
  Node* pull() {
```
7f8085,**7f8085**
```
    c = 1;
    if (l) c += l->c;
    if (r) c += r->c;
    return this;
  }
  void push() {}
};
```
6048e9,**39beab**

## Implicit Treap

**description:** Can easily be changed to a normal treap by changing split condition.
**time:** $O(\log n)$

```
template <class T> struct Treap {
  T* root = 0;
  deque<T> s;
  template <class V>
  T* make_node(V x) {
    s.emplace_back(x);
    return &s.back();
  }
```
27cc25,**27cc25**
```
  int cnt(T* v) {
    return v ? v->c : 0;
  }
  pair<T*, T*> split(T* v, int k) { // [0, k), [k, n)
    if (!v) return {};
    v->push();
    if (cnt(v->l) >= k) { // "n->x >= k" for set
      auto [L, R] = split(v->l, k);
```
c13926,**91c956**
```
      v->l = R;
      return {L, v->pull()};
    } else {
      auto [L, R] = split(v->r, k - cnt(v->l) - 1); // "k"
      v->r = L;
      return {v->pull(), R};
    }
  }
```
317f90,**8e1a7c**
```
  T* merge(T* l, T* r) {
    if (!l || !r) return l ? l : r;
    l->push(), r->push();
    if (l->y > r->y) {
      l->r = merge(l->r, r);
      return l->pull();
    } else {
      r->l = merge(l, r->l);
```
6f54e6,**67ce7e**
```
      return r->pull();
    }
  }
  void insert(T* v, int k) { // example
    auto [L, R] = split(root, k);
    root = merge(merge(L, v), R);
  }
};
```
9d79ca,**eeaf99**

## Mo Queries

**description:** Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change `step` to add/remove the edge $(a, c)$ and remove the initial `add` call (but keep `in`).
**time:** $O(n\sqrt{q})$

```
void add(int ind, int end); // add a[ind] (end = 0 or 1)
void del(int ind, int end); // remove a[ind]
int calc(); // compute current answer
vi mo(vec<array<int, 2>>& Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x[0]/blk, x[1] ^ -(x[0]/blk & 1))
  iota(all(s), 0);
```
ae0239,**ae0239**
```
  sort(all(s), [&](int s, int t){ return K(Q[s]) <
K(Q[t]); });
  for (int qi : s) {
    auto q = Q[qi];
    while (L > q[0]) add(--L, 0);
    while (R < q[1]) add(R++, 1);
    while (L < q[0]) del(L++, 0);
    while (R > q[1]) del(--R, 1);
    res[qi] = calc();
```
4d8363,**aafd2f**
```
  }
#undef K
  return res;
}
vi moTree(vec<array<int, 2>>& Q, vec<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
```
05c4d7,**f87a55**
```
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
```
329c88,**09fe43**
```
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk &
1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) <
K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
```
3b584a,**19983d**
```
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
#undef K
#undef step
```
f505f5,**ea1fa1**
```
  return res;
}
```
510bfa,**fbb9aa**

## Bit Vector

```
using ull = uint64_t;
struct BitVec {
  vec<pair<ull, int>> b;
  BitVec(const vec<bool>& a): b(sz(a) / 64 + 1) {
    rep(i,0,sz(a)) {
      auto& [x, y] = b[i >> 6];
      x |= ull(a[i]) << (i & 63), y += a[i];
    }
```
13be09,**13be09**
```
    rep(i,1,sz(b)) b[i].second += b[i - 1].second;
  }
  int rank(int r) {
    auto [x, y] = b[r >> 6];
    return y - popcount(x & -1ULL << (r & 63));
  }
};
```
53dda8,**bf1433**

## Wavelet Matrix

**description:** Supports quantile and rank queries in some range $[l, r)$.
**time:** $O(\log \sigma)$
**space:** $O(\frac{n \log \sigma}{64})$

```
struct Wavelet {
  int n;
  vec<BitVec> bv;
  Wavelet(vec<ull> a, ull sigma) :
    n(sz(a)), bv(bit_width(sigma), {{}}) {
    for (int h = sz(bv); h--;) {
      int i = 0;
      vec<bool> b(n);
      ranges::stable_partition(a,
        [&](ull x) { return b[i++] = (~x >> h) & 1; });
      bv[h] = b;
    }
  }
  ull kth(int l, int r, int k) {
    ll res = 0;
    for (int h = sz(bv); h--;) {
      int l0 = bv[h].rank(l), r0 = bv[h].rank(r);
      if (k < r0 - l0) l = l0, r = r0;
      else
        k -= r0 - l0, res |= 1ULL << h,
          l += bv[h].rank(n) - l0, r += bv[h].rank(n) - r0;
    }
    return res;
  }
  int count(int l, int r, ull ub) {
    int res = 0;
    for (int h = sz(bv); h--;) {
      int l0 = bv[h].rank(l), r0 = bv[h].rank(r);
      if ((~ub >> h) & 1) l = l0, r = r0;
      else
        res += r0 - l0, l += bv[h].rank(n) - l0,
          r += bv[h].rank(n) - r0;
    }
    return res;
  }
};
```

145cfe,**145cfe** ef8040,**d6efc5** 0c60da,**7b772b**

## Li Chao Lazy

**time:** Generally $O(\log n)$ except range updates $O(\log^2 n)$

```
struct LiChao {
  const int L = -1e9, R = 1e9;
  struct Line {
    ll a, b;
    ll la, lb; // lazy
    array<int, 2> ch;
    Line(ll a_ = 0, ll b_ = LINF) :
      a(a_), b(b_), la(0), lb(0), ch({-1, -1}) {}
    ll operator ()(ll x) { return a*x + b; }
  };
  vec<Line> ln;
  int ch(int p, int d) {
    if (ln[p].ch[d] == -1) {
      ln[p].ch[d] = ln.size();
      ln.emplace_back();
    }
    return ln[p].ch[d];
  }
  LiChao() { ln.emplace_back(); }
  void prop(int p, int l, int r) {
    if (ln[p].la == 0 and ln[p].lb == 0) return;
    ln[p].a += ln[p].la, ln[p].b += ln[p].lb;
    if (l != r) {
      int pl = ch(p, 0), pr = ch(p, 1);
      ln[pl].la += ln[p].la, ln[pl].lb += ln[p].lb;
      ln[pr].la += ln[p].la, ln[pr].lb += ln[p].lb;
    }
    ln[p].la = ln[p].lb = 0;
  }
  ll query(int x, int p=0, int l=L, int r=R) {
    prop(p, l, r);
    ll ret = ln[p](x);
    if (ln[p].ch[0] == -1 and ln[p].ch[1] == -1) return ret;
    int m = l + (r-l)/2;
    if (x <= m) return min(ret, query(x, ch(p, 0), l, m));
    return min(ret, query(x, ch(p, 1), m+1, r));
  }
  void push(Line s, int p, int l, int r) {
    prop(p, l, r);
    int m = l + (r-l)/2;
    bool L = s(l) < ln[p](l);
    bool M = s(m) < ln[p](m);
    bool R = s(r) < ln[p](r);
    if (M) swap(ln[p].a, s.a), swap(ln[p].b, s.b);
    if (s.b == LINF) return;
    if (L != M) push(s, ch(p, 0), l, m);
    else if (R != M) push(s, ch(p, 1), m+1, r);
  }
  void insert(Line s, int a=L, int b=R, int p=0, int l=L, int
r=R) {
    prop(p, l, r);
    if (a <= l and r <= b) return push(s, p, l, r);
    if (b < l or r < a) return;
    int m = l + (r-l)/2;
    insert(s, a, b, ch(p, 0), l, m);
    insert(s, a, b, ch(p, 1), m+1, r);
  }
  void shift(Line s, int a=L, int b=R, int p=0, int l=L, int
r=R) {
    prop(p, l, r);
    int m = l + (r-l)/2;
    if (a <= l and r <= b) {
      ln[p].la += s.a, ln[p].lb += s.b;
      return;
    }
    if (b < l or r < a) return;
    if (ln[p].b != LINF) {
      push(ln[p], ch(p, 0), l, m);
      push(ln[p], ch(p, 1), m+1, r);
      ln[p].a = 0, ln[p].b = LINF;
    }
    shift(s, a, b, ch(p, 0), l, m);
    shift(s, a, b, ch(p, 1), m+1, r);
  }
};
```

8d42e6,**b4d84e** 386d93,**aa3c87** 71c2ee,**46b90d** 23794b,**eb4075** c30a0f,**c393e4** 924984,**62a05f** 192b09,**9427f8** b2cf95,**5a2147** 2145c1,**4c6536**

# Number Theory

## Fast Modular Multiplication

**description:** Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$. `modmul` runs 2x faster than the naive `(__int128_t)a * b % M`.
**time:** $O(1)$ for `modmul`, $O(\log e)$ for `modpow`.

```
using ull = uint64_t;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}
```

879209,**778ed1** 94890c,**94890c**

## Miller Rabin

**description:** Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
**time:** $|A| \cdot O(\text{modpow})$

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

f7944d,**f7944d** e9b888,**60dcd1**

## Factor

**description:** Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. $2299 \to \{11, 19, 11\}$).
**time:** $O(n^{\frac{1}{4}})$

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
```

```
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
```
0b4d32,**0b4d32**
```
  return __gcd(prd, n);
}
vec<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
```
1c3f69,**eb9f43**
```
  return l;
}
```
a27b55,**6f52f8**

## Modular Arithmetic

```
const ll mod = 998244353; // use const!
#include "mod_pow.h"
struct Mod {
  ll x;
  Mod(ll x) : x(x % mod) {}
  operator ll() const { return x; }
  Mod operator+(Mod b) { return x + b.x; }
  Mod operator-(Mod b) { return x - b.x + mod; }
```
c20060,**c20060**
```
  Mod operator*(Mod b) { return x * b.x; }
  Mod operator/(Mod b) { return *this * b.inverse(); }
  Mod pow(ll e) {
    return modpow(x, e);
  }
  Mod inverse() {
    return pow(mod - 2);
  }
```
2758c4,**7fdc58**
```
};
```
2145c1,**296343**

## Extended Euclidean Algorithm

**description:** Finds two integers $x$ and $y$, such that $ax + by = \gcd(a, b)$. If $a$ and $b$ are coprime, then $x$ is the inverse of $a$ modulo $b$.
**time:** $O(\log \min(a, b))$

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```
33ba8f,**33ba8f**

## Morbius

**time:** $O(L \log L)$

```
const int L = 1e6;
array<int8_t, L> mu;
void calculateMu() {
  mu[1] = 1;
  rep(i,1,L) if (mu[i]) for (int j=2*i; j<L; j+=i) mu[j] -=
mu[i];
}
```
6f1e43,**6f1e43**

## CRT

**description:** Computes $x$ such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, $x$ will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
**time:** $O(\log n)$

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```
04d93a,**04d93a**

## Continued Fractions

**description:** Given $N$ and a real number $x \ge 0$, finds the closest rational approximation $\frac{p}{q}$ with $p, q \le N$. It will obey $|\frac{p}{q} - x| \le \frac{1}{qN}$.
**time:** $O(\log N)$

```
using d = double; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
       a = (ll)floor(y), b = min(a, lim),
       NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
```
d2e831,**d2e831**
```
      // if b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // return {P, Q} here for a more canonical
approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
```
360696,**284a47**
```
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```
f55260,**59018b**

## Fast Eratosthenes

**description:** Prime sieve for generating all primes smaller than LIM using segmented sieve. Despite its $n \log \log n$ complexity, segmented sieve is still faster than other options due to low memory usage.
**time:** LIM $= 10^9 \approx 1.5s$

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
```
97fea7,**97fea7**
```
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
```
7f7138,**3db15e**
```
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```
0163b1,**6b2912**

## Fractional Binary Search

**description:** Given $f$ and $N$, finds the smallest fraction $\frac{p}{q}$ in $[0, 1]$ such that $f\left(\frac{p}{q}\right)$ is true, and $p, q \le N$.
**time:** $O(\log N)$

```
struct Frac { ll p, q; };
template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
```
6623f0,**6623f0**
```
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }
    }
```
707740,**d35347**
```
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```
8b8a1a,**27ab3e**

## Modular Logarithm

**description:** Returns the smallest $x > 0$ such that $a^x = b \pmod{m}$, or $-1$ if no such $x$ exists. Uses baby-step giant-step method.
**time:** $O(\sqrt{m})$

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
```
457a32,**457a32**
```
      return n * i - A[e];
```

```
  return -1;
}
```
929168,**c040b8**

## Modular Square Root

**description:** Tonelli-Shanks algorithm for modular square roots. Finds $x$ such that $x^2 = a \pmod p$ ($-x$ gives the other solution).
**time:** $O(\log^2 p)$

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  if (modpow(a, (p-1)/2, p) != 1) return -1; // no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
```
d0f8bb,**d0f8bb**
```
  while (s % 2 == 0)
    ++r, s /= 2;
  // find a non-square mod p
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
```
c950ea,**f56250**
```
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
```
66c471,**7bd99b**
```
}
```
cbb184,**b7cab4**

## Modular Sum

**description:** Sums of mod'ed arithmetic progressions. $\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c \bmod m)$. divsum is similar but for floored division.
**time:** $O(\log m)$

```
using ull = uint64_t;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
/// ^ written in a weird way to deal with overflows correctly
ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
```
2e6f61,**2e6f61**
```
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```
066bdf,**37f2bb**

## Euler's Totient Function

**description:** Euler's phi function: $\varphi(n) =$ number of positive integers $\leq n$ that are coprime with $n$.
**time:** $O(\text{LIM} \log\log \text{LIM})$

```
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```
cf7d6d,**cf7d6d**

# Numerical

## Berlekamp Massey

**description:** Recovers any $n$-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms.
**time:** $O(N^2)$

```
vec<ll> berlekampMassey(vec<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vec<ll> C(n), B(n), T;
  C[0] = B[0] = 1;
  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
```
8b58c4,**8b58c4**
```
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }
  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
```
710147,**922b7c**
```
  return C;
}
```
c4e2ad,**558f8a**

## Polynomial

**description:** Basic polynomial operations including evaluation, differentiation, and root division.
**time:** $O(n)$

```
struct Poly {
  vec<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
```
31274f,**31274f**
```
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  }
```
3bdf10,**6c7a19**
```
};
```
2145c1,**864e12**

## Polynomial Interpolation

**description:** Given $n$ points $(x[i], y[i])$, computes an $n-1$-degree polynomial that passes through them.
**time:** $O(n^2)$

```
using vd = vec<double>;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
```
dcb720,**dcb720**
```
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```
003d15,**99ff48**

## Polynomial Roots

**description:** Finds the real roots to a polynomial using derivative analysis and binary search.
**time:** $O(n^2 \log(\frac{1}{\varepsilon}))$

```
vec<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vec<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
```
3361fe,**3361fe**
```
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
```
c9d2fb,**4cb75c**
```
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```
a7a8b6,**83db0d**

## Linear Recurrence

**description:** Generates the $k$'th term of an $n$-order linear recurrence. Faster than matrix multiplication.
**time:** $O(n^2 \log k)$

```
using Poly = vec<ll>;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);
  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
```
4c6066,**4c6066**
```
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
mod;
    res.resize(n + 1);
    return res;
  };
  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;
  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
```
b01bc4,**94f0c0**
```
    e = combine(e, e);
  }
  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```
b73f32,**3c8e2b**

## Integer Determinant

**description:** Calculates determinant using modular arithmetics. Can be used for pure-integer version by removing modulo operations.
**time:** $O(N^3)$

```
ll det(vec<vec<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
```
80eca6,**80eca6**
```
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
```
693fdb,**4465e1**
```
}
```
cbb184,**a89423**

## Tridiagonal

**description:** Solves tridiagonal systems of equations. Useful for problems of the form $a_i = b_i \cdot a_{i-1} + c_i \cdot a_{i+1} + d_i$.
**time:** $O(N)$

```
using T = double;
vec<T> tridiagonal(vec<T> diag, const vec<T>& super,
    const vec<T>& sub, vec<T> b) {
  int n = sz(b); vi tr(n);
  rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
```
7712d4,**7712d4**
```
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else {
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    }
  }
  for (int i = n; i--;) {
    if (tr[i]) {
```
72ff06,**5d1e08**
```
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else {
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    }
  }
```
9689aa,**768d3b**
```
  return b;
}
```
bec785,**3bc52a**

## Matrix Inverse (mod $p$)

**description:** Invert matrix $A$ modulo a prime. Returns rank; result is stored in $A$ unless singular (rank $< n$). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where $A^{-1}$ starts as the inverse of $A \bmod p$, and $k$ is doubled in each step.
**time:** $O(n^3)$

```
int matInv(vec<vec<ll>>& A) {
  int n = sz(A); vi col(n);
  vec<vec<ll>> tmp(n, vec<ll>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
      r = j; c = k; goto found;
```
01b187,**01b187**
```
    }
    return i;
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    ll v = modpow(A[i][i], mod - 2);
```
860b4c,**9eb9a8**
```
    rep(j,i+1,n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
    }
    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
```
7a05ed,**a64321**
```
    A[i][i] = 1;
  }
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    ll v = A[j][i];
    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
  }
  rep(i,0,n) rep(j,0,n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
```
712485,**060ed2**
```
  return n;
}
```
400ef4,**ed04e6**

## Solve Linear

**description:** Solves $Ax = b$ using Gaussian elimination. Returns rank, or $-1$ if no solutions exist.
**time:** $O(n^2 m)$

```
using vd = vec<double>;
const double eps = 1e-12;
int solveLinear(vec<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    double v, bv = 0;
```
495799,**495799**
```
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
```
cb5792,**c32caa**
```
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
```
c0cb45,**98c5da**
```
    }
    rank++;
  }
  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
```
3dd6b3,**da16ea**
```
  }
  return rank; // (multiple solutions if rank < m)
}
```
80760b,**5ec9e9**

## Fast Subset Transform

**description:** Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of $a$ must be a power of two.
**time:** $O(n \log n)$

```
void FST(auto& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      auto &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        // inv ? pii(v, u - v) : pii(u + v, u); // OR
        // pii(u + v, u - v);                   // XOR
    }
  }
  // if (inv) for (auto& x : a) x /= sz(a); // XOR only
}
auto conv(auto a, auto b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] = a[i] * b[i];
  FST(a, 1); return a;
}
```
6b9328,**6b9328** 58ed71,**e545f6**

## NTT

**time:** $O(n \log n)$

```
// the maximum length of the resulting convolution vector is
2^LG
const int LG = 23;
template<class u, class uu, u p, u root>
struct FFT {
  u r[1+(2<<LG)];
  constexpr u m(u a, u b) {
    uu k = uu(a)*b;
    #define op(g) g*(g*p+2)
    k += u(k) * (op(op(op(op(op(-p))))) * uu(p);
    #undef op
    return u(k>>(8*sizeof(u)));
  }
  constexpr u red(u k, u a) { return a-k*(a>=k); }
  FFT() {
    u k = r[2<<LG] = -p%p, b=root, e = p>>LG;
    for(; e; e/=2, b=m(b,b)) if(e%2) k=m(k, b);
    for(int i = (2<<LG) - 1; i >= 0; i--) r[i]=red(p,
m(r[i+1], k)), i&(i-1)?0:k=m(k,k);
    assert(r[2] != r[3]); assert(r[1] == r[2]);
  }
  vec<ll> cv(const vec<ll> &as, const vec<ll> &bs, u *v) {
    int c=max(sz(as)+sz(bs)-1, 0), n=1;
    assert(c <= (1<<LG));
    u h=u(uu(-p)*-p%p), a=m(h, p/2+1), x, y;
    while(n<c) n*=2, h=red(p, m(h, a));
    rep(i,0,n)
      v[i] = i<sz(as) ? u(as[i]) : 0,
      v[i+n] = i<sz(bs) ? u(bs[i]) : 0;
    for(auto s:{v,v+n})
    for(int j = n; j >= 2; j--) for(int k=j&-j; k/=2;)
```
d23dca,**d23dca** cde2ee,**eccbc5** 437e3a,**13ab88**

```
rep(i,j-k,j)
      x=s[i], y=s[i-k],
      s[i-k] = red(2*p, x+y),
      s[i] = m(2*p+y-x, r[3*k-j+i]);
    rep(i,0,n) v[i] = m(v[i], v[i+n]);
    rep(j,2,n+1) for(int k=1; !(k&j); k*=2) rep(i,j-k,j)
      x = m(v[i], r[3*k+j-i]),
      y = red(2*p, v[i-k]),
      v[i-k]=x+y, v[i]=2*p+y-x;
    rep(i,0,c) v[i] = red(p, m(v[i], h));
    return vec<ll>(v, v+c);
  }
};
// for modular convolutions modulo 998244353:
vec<ll> convSmall(const vec<ll> &as, const vec<ll> &bs) {
  static uint32_t v[2<<LG];
  static FFT<uint32_t, uint64_t, 998244353, 3> fft;
  return fft.cv(as, bs, v);
}
// for modular convolutions modulo a 62 bit prime:
vec<ll> convBig(const vec<ll> &as, const vec<ll> &bs) {
  static uint64_t v[2<<LG];
  static FFT<uint64_t, __uint128_t, (1ull<<62)-(18ull<<32)+1,
3> fft;
  return fft.cv(as, bs, v);
}
// for modular convolutions modulo an arbitrary 32-bit
modulus:
vec<ll> convSunzi(const vec<ll> &v1, const vec<ll> &v2, ll m)
{
  const uint64_t inv = 2703402103339935109ull,
    mod1 = (1ull<<62)-(18ull<<32)+1,
    mod2 = (1ull<<62)-(76ull<<32)+1;
  static_assert(__uint128_t(inv)*mod2%mod1 == (-mod1)%mod1);
  static uint64_t v[2<<LG];
  static FFT<uint64_t, __uint128_t, mod1, 3> fft1;
  static FFT<uint64_t, __uint128_t, mod2, 17> fft2;
  auto as=fft1.cv(v1, v2, v), bs=fft2.cv(v1, v2, v);
  rep(i,0,sz(as)) {
    auto d = fft1.m(mod1+as[i]-bs[i], inv);
    d -= mod1*(d >= mod1); d %= m;
    as[i] = (bs[i] + mod2%m*d)%m;
  }
  return as;
}
```
fb5404,**e68698** 267370,**e077a1** 9b60ae,**5751b5** 0d876f,**0fcec3** 55c52d,**0e24d2** 0c6a3f,**2940ee**

## Golden Section Search

**description:** Finds the argument minimizing the function $f$ in the interval $[a, b]$ assuming $f$ is unimodal on the interval.
**time:** $O\left(\log\left(\frac{b-a}{\varepsilon}\right)\right)$

```
// it is important for r to be precise, otherwise we don't
necessarily maintain the inequality a < x1 < x2 < b.
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { // change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```
012afe,**012afe** b19c7f,**31d45b**

## Hill Climbing

**description:** Poor man's optimization for unimodal functions. Finds local minimum by exploring neighboring points.
**time:** $O$(iterations)

```
using P = array<double, 2>;
template<class F> pair<double, P> hillClimb(P start, F f) {
  pair<double, P> cur(f(start), start);
  for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
      P p = cur.second;
      p[0] += dx*jmp;
      p[1] += dy*jmp;
      cur = min(cur, make_pair(f(p), p));
    }
  }
  return cur;
}
```
d019e9,**8b340f**

## Numerical Integration

**description:** Simple integration of a function over an interval using Simpson's rule. Error proportional to $h^4$.
**time:** $O(n)$

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
}
```
4756fc,**4756fc**

## Adaptive Numerical Integration

**description:** Fast integration using an adaptive Simpson's rule. Automatically adjusts precision.
**time:** $O$(adaptive)

```
using d = double;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
S2);
```
b0c6c3,**b0c6c3**

```
}
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```
136f39,**f57493**

## Simplex

**description:** Solves linear maximization problems: maximize $c^T x$ subject to $Ax \leq b, x \geq 0$.
**time:** $O(NM \cdot \#\text{pivots})$

```
using T = double; // long double, Rational, double + mod<P>...
using vd = vec<T>;
using vvd = vec<vd>;
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
s=j
struct LPSolver {
  int m, n;
```
85a9ca,**85a9ca**
```
  vi N, B;
  vvd D;
  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] =
b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
```
6825c8,**ede8b7**
```
    }
  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
```
9ca88f,**d44c2c**
```
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }
  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
```
419741,**2c0b2a**
```
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
```
1a6935,**9330ce**
```
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }
  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
```
f5dd87,**19506a**
```
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      }
```
256726,**7a915d**
```
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```
f42e93,**0ed7d8**

# Geometry

## Point

**description:** Class to handle points in the plane. `T` can be e.g. `double` or `ll`. (Avoid `int`.)

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template <class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) <
tie(p.x,p.y); }
  bool operator==(P p) const { return
tie(x,y)==tie(p.x,p.y); }
```
3a27ca,**3a27ca**
```
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
```
374c23,**e7b843**
```
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
```
1cfa6b,**91d8d5**
```
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```
ad487a,**47ec0a**

## Line Distance

**description:** Returns the signed distance between point `p` and the line containing points `a` and `b`. Positive value on left side and negative on right as seen from `a` towards `b`. `a == b` gives `nan`. `P` is supposed to be `Point<T>` or `Point3D<T>` where `T` is e.g. `double` or `ll`. It uses products in intermediate steps so watch out for overflow if using `int` or `ll`. Using `Point3D` will always give a non-negative distance. For `Point3D`, call `.dist` on the result of the cross product.

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```
f6bf6b,**f6bf6b**

## Segment Distance

**description:** Returns the shortest distance between point $p$ and the line segment from point $s$ to $e$.

```
using P = Point<double>;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```
7b3133,**7b3133**

## Line Intersection

**description:** If a unique intersection point of the lines going through `a,b` and `c,d` exists $\{1, \text{point}\}$ is returned. If no intersection point exists $\{0, (0,0)\}$ is returned and if infinitely many exists $\{-1, (0,0)\}$ is returned. The wrong position will be returned if `P` is `Point<ll>` and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using `int` or `ll`.

```
template<class P>
pair<int, P> lineInter(P a, P b, P c, P d) {
  auto d = (b - a).cross(d - c);
  if (d == 0) // if parallel
    return {-(a.cross(b, c) == 0), P(0, 0)};
  auto p = c.cross(b, d), q = c.cross(d, a);
  return {1, (a * p + b * q) / d};
}
```
bc3dd4,**bc3dd4**

## On Segment

**description:** Returns true iff $p$ lies on the line segment from $s$ to $e$.

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```
c597e8,**c597e8**

## Segment Intersection

**description:** If a unique intersection point between the line segments going from $a$ to $b$ and from $c$ to $d$ exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if `P` is `Point<ll>` and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using `int` or `ll`.

```
template<class P> vec<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
```

b6eb28,**b6eb28**

```
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

395ed6,**08ced5**

## Side Of

**description:** Returns where $p$ is as seen from $s$ towards $e$. 1/0/-1 $\leftrightarrow$ left/on line/right. If the optional argument `eps` is given 0 is returned if $p$ is within distance `eps` from the line. It uses products in intermediate steps so watch out for overflow if using `int` or `ll`.

```
template<class P>
ll sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
template<class P>
ll sideOf(const P& s, const P& e, const P& p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

89ff34,**89ff34**

## Linear Transformation

**description:** Apply the linear transformation (translation, rotation and scaling) which takes line $a - b$ to line $c - d$ to point $r$.

```
using P = Point<double>;
P linearTransformation(const P& a, const P& b,
    const P& c, const P& d, const P& r) {
  P dp = b-a, dq = d-c, num(dp.cross(dq), dp.dot(dq));
  return c + P((r-a).cross(num), (r-a).dot(num))/dp.dist2();
}
```

eadeb7,**eadeb7**

## Closest Pair

**description:** Finds the closest pair of points.
**time:** $O(n \log n)$

```
using P = Point<ll>;
array<P, 2> closest(vec<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, array<P, 2>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
```

494406,**494406**

```
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
```

10c9c4,**d19dfc**

```
}
```

cbb184,**f54944**

## Convex Hull

**description:** Returns a vector of the points of the convex hull in **ccw** order. Points on the edge of the hull between two other points are not considered part of the hull.
**time:** $O(n \log n)$

```
using P = Point<ll>;
vec<P> convexHull(vec<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vec<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
```

bb57f4,**bb57f4**

```
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] ==
h[1])};
}
```

e86333,**255bac**

## Manhattan MST

**description:** Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = |p.x - q.x| + |p.y - q.y|$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
**time:** $O(N \log N)$

```
using P = Point<int>;
vec<array<int, 3>> manhattanMST(vec<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vec<array<int, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
```

e796bc,**e796bc**

```
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
```

257d9d,**2a0843**

```
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x,
p.y);
  }
  return edges;
}
```

064dd2,**0e81ef**

## Circumcircle

**description:** The circumcirle of a triangle is the circle intersecting all three vertices. `ccRadius` returns the radius of the circle going through points $A$, $B$ and $C$ and `ccCenter` returns the center of the same circle.

```
using P = Point<double>;
double ccRadius(const P& A, const P& B, const P& C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(const P& A, const P& B, const P& C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
```

f1e2a0,**f1e2a0**

```
}
```

cbb184,**ccf7fd**

## Minimum Enclosing Circle

**description:** Computes the minimum circle that encloses a set of points.
**time:** Expected $O(n)$

```
auto mec(vec<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
```

4f560b,**4f560b**

```
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return pair{o, r};
```

78fb20,**09f0f2**

```
}
```

cbb184,**91bf54**

## Minkowski

**description:** Compute Minkowski sum of two strictly convex non empty polygons (i.e. two hulls). Returns answer in **ccw** order.
**time:** $O(n \log n + m \log m)$

```
void reorder(vec<P> &p) {
  if (sideOf(p[0], p[1], p[2]) < 0) reverse(all(p));
  rotate(p.begin(), min_element(all(p)), p.end());
}
vec<P> minkowskiSum(vec<P> p, vec<P> q) {
  if (min(sz(p), sz(q)) < 3) {
    vec<P> v;
    for (P pp : p) for (P qq : q) v.pb(pp + qq);
    return convexHull(v);
  }
  reorder(p), reorder(q);
  rep(i,0,2) p.pb(p[i]), q.pb(q[i]);
  vec<P> r;
  ll i = 0, j = 0;
  while (i + 2 < sz(p) || j + 2 < sz(q)) {
    r.pb(p[i] + q[j]);
    ll cross = (p[i + 1] - p[i]).cross(q[j + 1] - q[j]);
    i += cross >= 0, j += cross <= 0;
  }
  return r;
}
```
e28127,**e28127** f18450,**f7d44a** baeb21,**129a7b**

## Halfplane Intersection

**description:** Computes the intersection of a set of half-planes. Input is given as a set of planes, facing left. The intersection must form a convex polygon or be empty. Output is the convex polygon representing the intersection in **ccw** order. The points may have duplicates and be collinear.
**time:** $O(n \log n)$

```
using P = Point<double>;
struct Line {
  P p, q;
  double a;
  Line() {}
  Line(P p, P q) : p(p), q(q), a((q - p).angle()) {}
  bool operator<(Line o) const { return a < o.a; }
};
vec<P> halfPlaneIntersection(vec<Line> v) {
  sort(all(v));
  ll n = sz(v), q = 1, h = 0;
  const double eps = 1e-9;
  vec<Line> c(n+2);
#define L(a) a.p, a.q
#define PQ(a) (a.q - a.p)
#define I(j, k) lineInter(L(c[j]), L(c[k])).snd
  rep(i,0,n) {
    while (q < h && sideOf(L(v[i]), I(h, h-1), eps) < 0) h--;
    while (q < h && sideOf(L(v[i]), I(q, q+1), eps) < 0) q++;
    c[++h] = v[i];
    if (q < h && abs(PQ(c[h]).cross(PQ(c[h-1]))) < eps) {
      if (PQ(c[h]).dot(PQ(c[h-1])) <= 0) return {};
      if (sideOf(L(v[i]), c[--h].p, eps) < 0) c[h] = v[i];
    }
  }
  while (q < h - 1 && sideOf(L(c[q]), I(h, h-1), eps) < 0)
h--;
  while (q < h - 1 && sideOf(L(c[h]), I(q, q+1), eps) < 0) q+
+;
  if (h - q <= 1) return {};
  c[++h] = c[q];
  vec<P> s;
  rep(i,q,h) s.pb(I(i, i+1));
  return s;
}
#undef L
#undef PQ
#undef I
}
```
d64e5c,**a0fa96** 1e57e0,**ee64fc** 0c45a5,**50f8d1** 2fd167,**314835**

## Fast Delaunay Triangulation

**description:** Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Returns triangles in order $\{t[0][0], t[0][1], t[0][2], t[1][0], ...\}$, all counter-clockwise.
**time:** $O(n \log n)$

```
using P = Point<ll>;
typedef struct Quad* Q;
using lll = __int128_t; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point
struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;
bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r-
>r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}
pair<Q,Q> rec(const vec<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;
#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}
vec<P> triangulate(vec<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vec<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p);
\
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```
06515c,**9c9e81** 691292,**add3d8** 422f06,**4a02c4** 683611,**b09cca** eca5f3,**619dec** 54a694,**9433b3** 50fb79,**751bf1**

## Polygon Area

**description:** Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using `int` as T!

```
template<class T>
T polygonArea2(vec<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
```

```
  return a;
}
```
6771cc,**6771cc**

## Inside Polygon

**description:** Returns true if a point lies within the polygon. If strict is true, it returns false for points on the boundary.
**time:** $O(n)$

```
template<class P>
bool inPolygon(vec<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```
637faa,**637faa** 1d69d2,**0f07cd**

## Polygon Cut

**description:** Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
**time:** $O(N)$

```
typedef Point<double> P;
vec<P> polygonCut(const vec<P>& poly, P s, P e) {
  vec<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```
879b81,**879b81** 4b3b24,**f15ff3**

## Polygon Center

**description:** Returns the center of mass for a polygon.
**time:** $O(N)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```
9d5722,**9d5722** cbb184,**9706dc**

## Polygon Union

**description:** Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
**time:** $O(N^2)$

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vec<vec<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vec<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0)
{
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, 0.0),
1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```
24b9ff,**24b9ff** 9292e3,**f1f570** eade3d,**f06a18** b6d0bf,**6c4f4c** cbb184,**027ae3**

## Line-Hull Intersection

**description:** Line-convex polygon intersection. The polygon must be counter-clockwise and have no collinear points. Returns a pair describing the intersection of a line with the polygon. The points are returned in the same order as the line hits the polygon.
**time:** $O(\log N)$

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-
poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vec<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}
template <class P>
array<int, 2> lineHull(P a, P b, vec<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```
69add2,**69add2** 85c369,**333d21** 0db5b2,**26ebb6** 0a29a1,**3eed5d** 674e46,**827845**

## Circle Polygon Intersection

**description:** Returns the area of the intersection of a circle with a ccw polygon.
**time:** $O(n)$

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vec<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/
d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```
7554dd,**7554dd** cdd84d,**2927c1** f3ae2b,**870f7b**

# 3D Geometry

## Point 3D

**description:** Class to handle points in 3D space. `T` can be e.g. `double` or `ll`. (Avoid `int`.)

```
template <class T>
struct Point3D {
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
  explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
  bool operator<(R p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); }
```
af5a46,**af5a46**
```
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
  P cross(R p) const {
```
b8ad04,**a9fb7d**
```
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
  }
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  // azimuthal angle (longitude) to x-axis in interval [-pi,
pi]
  double phi() const { return atan2(y, x); }
  // zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*y),z); }
```
792bb1,**c1e43f**
```
  P unit() const { return *this/(T)dist(); } //makes dist()=1
  // returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  // returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  }
```
567952,**6c6b0d**
```
};
```
2145c1,**8058ae**

## Hull 3D

**description:** Computes all faces of the 3-dimension hull of a point set. **No four points must be coplanar**, or else random results will be returned. All faces will point outwards.
**time:** $O(n^2)$

```
typedef Point3D<double> P3;
struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
};
struct F { P3 q; int a, b, c; };
```
538b68,**538b68**
```
vec<F> hull3d(const vec<P3>& A) {
  assert(sz(A) >= 4);
  vec<vec<PR>> E(sz(A), vec<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vec<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
```
f3cfb0,**886eb9**
```
      q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  };
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);

```
bf2c83,**47043a**
```
  rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
      F f = FS[j];
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
```
1b9f59,**d59076**
```
        FS.pop_back();
      }
    }
    int nw = sz(FS);
    rep(j,0,nw) {
      F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i,
f.c);
      C(a, b, c); C(a, c, b); C(b, c, a);
```
b1f3e9,**4e12e0**
```
    }
  }
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
  return FS;
};
```
93ab99,**c17344**

## Polyhedron Volume

**description:** Magic formula for the volume of a polyhedron. Faces should point outwards.

```
template <class V, class L>
double signedPolyVolume(const V& p, const L& trilist) {
  double v = 0;
  for (auto i : trilist) v +=
p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v / 6;
}
```
3058c3,**3058c3**

## Spherical Distance

**description:** Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) `f1` ($\varphi_1$) and `f2` ($\varphi_2$) from $x$ axis and zenith angles (latitude) `t1` ($\theta_1$) and `t2` ($\theta_2$) from $z$ axis ($0 =$ north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. `dx*radius` is then the difference between the two points in the $x$ direction and `d*radius` is the total distance between the points.

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```
611f07,**611f07**

## Spherical Segment Intersection

Given non-antipodal endpoints $p_1, p_2$ and $q_1, q_2$ on the unit sphere ($|p_1| = |p_2| = |q_1| = |q_2| = 1$).
Parametrise the cones spanned by each geodesic:

$$p(s,t) = sp_1 + tp_2,\ s,t \geq 0$$

$$q(u,v) = uq_1 + vq_2,\ u,v \geq 0$$

The spherical geodesics intersect $\Leftrightarrow$ the cones intersect in a nontrivial ray $\Leftrightarrow$ there exist $s,t,u,v \geq 0$, not all zero, with $sp_1 + tp_2 = uq_1 + vq_2$. This gives a linear system (3 equations, 4 unknowns). Solve for $(s,t,u,v)$.
Accept intersection iff a solution exists with $(s > 0 \vee t > 0)$ and $(u > 0 \vee v > 0)$.

# Strings

## Polynomial Hashing

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a
problem.
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
```
80cf70,**80cf70**
```
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
```
3a5a2c,**40d284**
```
static const H C = (ll)1e11+3; // (order ~ 3e9; random also
```

```
ok)
struct HashInterval {
  vec<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};
vec<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vec<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}
H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return
h;}
```

2afc56,**d5f9e2** 8c5c08,**5130d8** 509050,**9a868a** 553d92,**9e9c8c**

## Trie

```
struct Trie {
  const int mn = 'A';
  struct Node {
    array<int, 26> next;
    bool end_of_word = 0;
    Node() { fill(begin(next), end(next), -1); }
  };
  deque<Node> t;
  Trie(): t(1) {}
  void insert(const string& s) {
    int v = 0;
    for (char ch : s) {
      int u = ch - mn;
      if (t[v].next[u] == -1) {
        t[v].next[u] = ssize(t);
        t.emplace_back();
      }
      v = t[v].next[u];
    }
    t[v].end_of_word = 1;
  }
  int find(const string& s) {
    int v = 0;
    for (char ch : s) {
      int u = ch - mn;
      if (t[v].next[u] == -1) return 0;
      v = t[v].next[u];
    }
    return t[v].end_of_word;
  }
};
```

74ef28,**74ef28** d2140c,**30d936** df6e1e,**29649d** b6fb06,**bc9a31**

## Suffix Array

**description:** Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is $i$-th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any nul chars.
**time:** $O(n \log n)$

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = sz(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p)
{
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

69d1c4,**69d1c4** 2e1c56,**9670e1** 011c3d,**635552**

## Suffix Tree

**description:** Ukkonen's algorithm for online suffix tree construction. Each node contains indices $[l, r)$ into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining $[l, r)$ substrings. The root is 0 (has $l = -1, r = 0$), non-existent children are $-1$. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).
**time:** $O(26N)$

```
struct SuffixTree {
  enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
  int toi(char c) { return c - 'a'; }
  string a; // v = cur node, q = cur position
  int t[N][ALPHA],l[N],r[N],p[N],s[N],v=0,q=0,m=2;
  void ukkadd(int i, int c) { suff:
    if (r[v]<=q) {
      if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
        p[m++]=v; v=s[v]; q=r[v];  goto suff; }
      v=t[v][c]; q=l[v];
    }
    if (q==-1 || c==toi(a[q])) q++; else {
      l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q;
      p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v;
      l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;
      v=s[p[m]];  q=l[m];
      while (q<r[m]) { v=t[v][toi(a[q])];  q+=r[v]-l[v]; }
      if (q==r[m])  s[m]=v;  else s[m]=m+2;
      q=r[v]-(q-r[m]);  m+=2;  goto suff;
    }
  }
  SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
  }
  // example: find longest common substring (uses ALPHA = 28)
  pii best;
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
      best = max(best, {len, r[node] - len});
    return mask;
  }
  static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
  }
};
```

f7dbb8,**f7dbb8** 99e069,**d6dde8** b82da4,**cb23ac** 5568af,**66dfb7** 740c82,**2dd106** 94ac82,**aae0b8**

## Suffix Automaton

**description:** Exactly all substrings of $s$ appear in states (each state represents one or more substrings); if $t$ is the longest string of state $v$ then `v.len = |t|` and every string in $v$ occurs only as a suffix of $t$ (the other suffixes of $t$ are found by following `v.l`), $p$ is the state for $s$ so terminal states lie on the suffix-link path from $p$ to the root, suffix links form the suffix tree of reversed $s$; automaton size $\leq 2N$ states and $\leq 3N$ transitions, extend amortized (adds 1–2 states, may change many links). Use `T = ll` for large alphabets, reserve $2N$ and prefer vector/unordered_map over map for speed.
**time:** $O(N \log K)$

```
template <class T = char> struct SuffixAutomaton {
  struct State { ll len = 0, l = -1; map<T, ll> t; };
  vector<State> s{1}; ll p = 0;
  void extend(T c) {
    ll k = SZ(s), q; s.pb({s[p].len+1});
    for(;p != -1 && !s[p].t.count(c); p = s[p].l)s[p].t[c] =
k;
```

```
    if (p == -1) s[k].l = 0;
    else if (s[p].len + 1 == s[q = s[p].t[c]].len) s[k].l = q;
```
f69ded,**f69ded**
```
    else {
      s.pb(s[q]), s.back().len = s[p].len + 1;
      for (; p!=-1 && s[p].t[c]==q; p=s[p].l) s[p].t[c] = k+1;
      s[q].l = s[k].l = k+1;
    }
    p = k;
  }
};
```
104647,**ada074**

## KMP

**description:** `pi[x]` computes the length of the longest prefix of `s` that ends at `x`, other than `s[0...x]` itself (`abacaba` → `0010123`). Can be used to find all occurrences of a string.
**time:** $O(n)$

```
vi pi(const string &s) {
  vi p(sz(s));
  for (int i = 1; i < sz(s); i++) {
    int g = p[i - 1];
    while (g && s[i] != s[g])
      g = p[g - 1];
    p[i] = g + (s[i] == s[g]);
  }
```
c86f43,**c86f43**
```
  return p;
}
vi match(const string &s, const string &pat) {
  vi p = pi(pat + '\0' + s), res;
  for (int i = sz(p) - sz(s); i < sz(p); i++) {
    if (p[i] == sz(pat))
      res.push_back(i - 2 * sz(pat));
  }
```
f15f03,**3a1336**
```
  return res;
}
```
510bfa,**a65d0d**

## Z-function

**description:** `z[i]` computes the length of the longest common prefix of `s[i:]` and s, except `z[0] = 0`. (`abacaba` → `0010301`)
**time:** $O(n)$

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
```
18e9a9,**18e9a9**
```
      l = i, r = i + z[i];
  }
  return z;
}
```
7fb6e6,**ee09e2**

## Manacher

**description:** For each position in a string, computes `p[0][i]` = half length of longest even palindrome around pos $i$, `p[1][i]` = longest odd (half rounded down).
**time:** $O(n)$

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
```
508df3,**508df3**
```
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```
75e0d2,**e7ad79**

## Aho Corasick

**description:** Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. `find(word)` returns for each position the index of the longest word that ends there, or $-1$ if none. `findAll(-, word)` finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
**time:** construction takes $O(26N)$, where $N =$ sum of length of patterns. `find(x)` is $O(N)$, where $N =$ length of $x$. `findAll` is $O(NM)$.

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vec<Node> N;
```
508ff4,**508ff4**
```
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
      else n = m;
```
92fb08,**44a923**
```
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vec<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
```
dd63c6,**f33843**
```
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
```
723993,**ced125**
```
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
```
6fb438,**b6b11c**
```
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
```
d6a5cf,**d6f65e**
```
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vec<vi> findAll(vec<string>& pat, string word) {
    vi r = find(word);
    vec<vi> res(sz(word));
```
f9a5d7,**56adaa**
```
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
```
7476a6,**a69397**
```
  }
};
```
e0360a,**d6c0b8**

# Graphs

## Johnson

**time:** $O(V^2 \log V + EV)$

1. First, a new node $q$ is added to the graph, connected by zero-weight edges to each of the other nodes.
2. Second, the Bellman–Ford algorithm is used, starting from the new vertex $q$, to find for each vertex $v$ the minimum weight $h(v)$ of a path from $q$ to $v$. If this step detects a negative cycle, the algorithm is terminated.

3. Next the edges of the original graph are reweighted using the values computed by the Bellman–Ford algorithm: an edge from $u$ to $v$, having length $w(u,v)$, is given the new length $w(u,v)+h(u)-h(v)$.
4. Finally, $q$ is removed, and Dijkstra's algorithm is used to find the shortest paths from each node $s$ to every other vertex in the reweighted graph. The distance in the original graph is then computed for each distance $D(u,v)$, by adding $h(v)-h(u)$ to the distance returned by Dijkstra's algorithm.

## Topological Sort

**time:** $O(V+E)$

```
vi topoSort(const vec<vi>& gr) {
  vi indeg(sz(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
  rep(j,0,sz(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```
35e163,**35e163**

## Strongly Connected Components

**description:** Finds strongly connected components in a directed graph. If vertices $u, v$ belong to the same component, we can reach $u$ from $v$ and vice versa. Returns the number of components and the component of each vertex. Natural order of components is reverse topological order (a component only has edges to components with lower index).

**time:** $O(V+E)$

```
auto scc(const auto& g) {
  int n = sz(g), num_sccs = 0, q = 0, s = 0;
  vi scc_id(n, -1), tin(n), st(n);
  auto dfs = [&](auto&& self, int v) -> int {
    int low = tin[v] = ++q;
    st[s++] = v;
    for (int u : g[v])
      if (scc_id[u] < 0)
```
cc2095,**cc2095**
```
        low = min(low, tin[u] ?: self(self, u));
    if (tin[v] == low) {
      while (scc_id[v] < 0) scc_id[st[--s]] = num_sccs;
      num_sccs++;
    }
    return low;
  };
  rep(i,0,n)
```
ded704,**bc7f84**
```
    if (!tin[i]) dfs(dfs, i);
  return pair{num_sccs, scc_id};
}
```
d9a0c3,**4f34e7**

## Bridges

**description:** Finds bridges in a graph. `g` is an adjacency list where each value is of type `{v, i} ∈ g[u]` where `i` is the index of the edge.

**time:** $O(V+E)$

```
auto bridges(const auto& g, int m) {
  int n = sz(g), num_ccs = 0, q = 0, s = 0;
  vi br_id(n, -1), is_br(m), tin(n), st(n);
  auto dfs = [&](auto&& self, int v, int p) -> int {
    int low = tin[v] = ++q;
    st[s++] = v;
    for (auto [u, e] : g[v])
      if (e != p && br_id[u] < 0)
```
18f1d3,**18f1d3**
```
        low = min(low, tin[u] ?: self(self, u, e));
    if (tin[v] == low) {
      if (p != -1) is_br[p] = 1;
      while (br_id[v] < 0) br_id[st[--s]] = num_ccs;
      num_ccs++;
    }
    return low;
  };
  rep(i,0,n)
    if (!tin[i]) dfs(dfs, i, -1);
  return tuple{num_ccs, br_id, is_br};
}
```
79e610,**2eb7d1**
19a3af,**0d169e**

## Cuts

**description:** Finds cuts/articulation points in a graph. `g` is an adjacency list where each value is of type `{v, i} ∈ g[u]` where `i` is the index of the edge.

**time:** $O(V+E)$

```
auto cuts(const auto& adj, int m) {
  int n = sz(adj), num_bccs = 0, q = 0, s = 0;
  vi bcc_id(m, -1), is_cut(n), tin(n), st(m);
  auto dfs = [&](auto&& self, int v, int p) -> int {
    int low = tin[v] = ++q;
    for (auto [u, e] : adj[v]) {
      assert(v != u);
      if (e == p) continue;
```
584e9b,**584e9b**
```
      if (tin[u] < tin[v]) st[s++] = e;
      int lu = -1;
      low = min(low, tin[u] ?: (lu = self(self, u, e)));
      if (lu >= tin[v]) {
        is_cut[v] = p >= 0 || tin[v] + 1 < tin[u];
        while (bcc_id[e] < 0) bcc_id[st[--s]] = num_bccs;
        num_bccs++;
      }
```
45a9fd,**a3f48a**
```
    }
    return low;
  };
  rep(i,0,n)
    if (!tin[i]) dfs(dfs, i, -1);
  return tuple{num_bccs, bcc_id, is_cut};
}
```
8fce09,**f9ff5a**

## 2-SAT

**time:** $O(n+m)$

```
struct TwoSat {
  int N;
  vec<vi> gr;
  vi values; // 0 = false, 1 = true
  TwoSat(int n = 0) : N(n), gr(2*n) {}
  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
```
99f40a,**99f40a**
```
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }
  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
```
8f8e6d,**42d36a**
```
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
```
acb359,**a31cb9**
```
  }
  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
```
684253,**77e127**
```
  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }
  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
```
db1468,**734437**
```
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
```
6b1488,**eb77f4**
```
  }
};
```
e0360a,**25dcb3**

## Lowest Common Ancestor

**description:** Finds lowest common ancestor with a sparse table.

**time:** $O(1)$

**memory:** $O(n \log n)$

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  SparseTable<int> rmq;
```

```
  LCA(vec<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
  void dfs(vec<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }
  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
};
```
f49329,**f49329** aca3e1,**9ca671** e0360a,**60c26d**

## Binary Lifting

**description:** Can be augmented to solve path queries without inverse.
**time:** $O(\log n)$
**memory:** $O(n)$

```
struct Lift {
  vi d, p, j;
  Lift(vec<vi>& C) : d(sz(C)), p(d), j(d) { dfs(C, 0); }
  void dfs(vec<vi>& C, int u) {
    int jmp = (d[u] + d[j[j[u]]] == 2 * d[j[u]]) ? j[j[u]] :
u;
    for (int v : C[u]) if (v != p[u])
      d[v] = d[p[v] = u] + 1, j[v] = jmp, dfs(C, v);
  }
  int lca(int u, int v) {
    if (d[u] < d[v]) swap(u, v);
    while (d[u] > d[v]) u = d[j[u]] >= d[v] ? j[u] : p[u];
    if (u == v) return u;
    while (p[u] != p[v]) {
      if (j[u] != j[v]) u = j[u], v = j[v];
      else u = p[u], v = p[v];
    }
    return p[u];
  }
  int kth(int u, int k) {
    if (k > d[u]) return -1;
    k = d[u] - k;
    while (d[u] > k) u = d[j[u]] >= k ? j[u] : p[u];
    return u;
  }
};
```
9bb257,**9bb257** 324218,**7e7b5e** 5c7114,**decd1b** 2145c1,**d57630**

## Heavy Light Decomposition

**time:** $O(\log^2 N)$
**description:** Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log n$ light edges. Root must be node 0

```
template <bool VALS_EDGES, class T> struct HLD {
  int N, time = 0;
  vec<vi> adj;
  vi par, siz, rt, pos;
  T st;
  HLD(vec<vi> adj) : N(sz(adj)), adj(adj),
    par(N, -1), siz(N, 1), rt(N), pos(N), st(N) {
      dfsSz(0); dfsHld(0);
  }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
      adj[u].erase(find(all(adj[u]), v));
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = time++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  template <class B> void process(int u, int v, B op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op(pos[rt[v]], pos[v] + 1);
    }
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, auto val) {
    process(u, v, [&](int l, int r) { st.upd(l, r, val); });
  }
  auto queryPath(int u, int v) { // modify depending on
problem
    ll res = 0;
    process(u, v, [&](int l, int r) {
      res += st.query(l, r);
    });
    return res;
  }
  auto querySubtree(int v) { // modifySubtree is similar
    return st.query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
  }
};
```
d19517,**bd2131** e97acc,**25ed48** faaee7,**820989** 9ef0e5,**5d114f** 5daace,**ee50ca**

## Link Cut Tree

**time:** Amortized $O(\log N)$

```
struct SplayTree {
  struct Node {
    int ch[2] = {0, 0}, p = 0;
    ll self = 0, path = 0;        // Path aggregates
    ll sub = 0, vir = 0;          // Subtree aggregates
    bool flip = 0;                // Lazy tags
  };
  vec<Node> T;
  SplayTree(int n) : T(n + 1) {}
  void push(int x) {
    if (!x || !T[x].flip) return;
    int l = T[x].ch[0], r = T[x].ch[1];
    T[l].flip ^= 1, T[r].flip ^= 1;
    swap(T[x].ch[0], T[x].ch[1]);
    T[x].flip = 0;
  }
  void pull(int x) {
    int l = T[x].ch[0], r = T[x].ch[1]; push(l); push(r);
    T[x].path = T[l].path + T[x].self + T[r].path;
    T[x].sub = T[x].vir + T[l].sub + T[r].sub + T[x].self;
  }
  void set(int x, int d, int y) {
    T[x].ch[d] = y; T[y].p = x; pull(x);
  }
  void splay(int x) {
    auto dir = [&](int x) {
      int p = T[x].p; if (!p) return -1;
      return T[p].ch[0] == x ? 0 : T[p].ch[1] == x ? 1 : -1;
    };
    auto rotate = [&](int x) {
      int y = T[x].p, z = T[y].p, dx = dir(x), dy = dir(y);
      set(y, dx, T[x].ch[!dx]);
      set(x, !dx, y);
      if (~dy) set(z, dy, x);
      T[x].p = z;
    };
    for (push(x); ~dir(x); ) {
      int y = T[x].p, z = T[y].p;
      push(z); push(y); push(x);
      int dx = dir(x), dy = dir(y);
      if (~dy) rotate(dx != dy ? x : y);
      rotate(x);
    }
  }
};
struct LinkCut : SplayTree {
  LinkCut(int n) : SplayTree(n) {}
  int access(int x) {
    int u = x, v = 0;
    for (; u; v = u, u = T[u].p) {
      splay(u);
      int& ov = T[u].ch[1];
      T[u].vir += T[ov].sub;
      T[u].vir -= T[v].sub;
      ov = v; pull(u);
    }
    return splay(x), v;
  }
  void reroot(int x) {
    access(x); T[x].flip ^= 1; push(x);
  }
  void link(int u, int v) {
    reroot(u); access(v);
    T[v].vir += T[u].sub;
    T[u].p = v; pull(v);
  }
  void cut(int u, int v) {
```
40fae5,**36592b** 03df6b,**ff6636** 5a797f,**d20e52** ef771c,**9de971** d2bc76,**04ed38** 531ed1,**f39f6c** fe475f,**f78e78** 41055c,**41055c** 

```
    reroot(u); access(v);
    T[v].ch[0] = T[u].p = 0; pull(v);
  }
  // rooted tree LCA. Returns 0 if u and v aren't connected.
  int lca(int u, int v) {
```
719311,**7da4f6**
```
    if (u == v) return u;
    access(u); int ret = access(v);
    return T[u].p ? ret : 0;
  }
  // query subtree of u where v is outside the subtree.
  ll subtree(int u, int v) {
    reroot(v); access(u); return T[u].vir + T[u].self;
  }
```
351d3c,**c88fb1**
```
  // query path [u..v]
  ll path(int u, int v) {
    reroot(u); access(v); return T[v].path;
  }
  // update vertex u with value v
  void update(int u, ll v) {
    access(u); T[u].self = v; pull(u);
  }
```
2ebb27,**be7711**
```
};
```
2145c1,**20af44**

## Centroid Tree

**description:** Calculate the centroid tree of a tree. Returns vector of fathers ($-1$ for root).
**time:** $O(N \log N)$

```
vi centroidTree(vec<vi>& g) {
  int n = sz(g);
  vi vis(n), P(n), S(n);
  auto calc_sz = [&](auto&& self, int v, int p) -> int {
    S[v] = 1;
    for (int y : g[v]) if (y != p && !vis[y])
      S[v] += self(self, y, v);
    return S[v];
```
c4cbdb,**c4cbdb**
```
  };
  auto dfs = [&](auto&& self, int v, int p, int s) -> void {
    if (s < 0) s = calc_sz(calc_sz, v, -1);
    for (int y : g[v]) if (!vis[y] && S[y] * 2 >= s) {
      S[v] = 0;
      self(self, y, p, s);
      return;
    }
```
8e86c3,**90045b**
```
    vis[v] = 1;
    P[v] = p;
    for (int y : g[v]) if (!vis[y]) self(self, y, v, -1);
  };
  dfs(dfs, 0, -1, -1);
  return P;
}
```
67875b,**1ca179**

## Euler Walk

**description:** Eulerian undirected/directed path/cycle algorithm. Input should be a vec of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add `.second` to `s` and `ret`.
**time:** $O(V + E)$

```
vi eulerWalk(vec<vec<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back();
continue; }
    tie(y, e) = gr[x][it++];
```
8e6508,**8e6508**
```
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
}
```
a3a0af,**b6fb45**

## Maximum Clique

**description:** Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
**time:** Runs in about $1s$ for $n = 155$ and worst case random graphs ($p = .90$). Runs faster for sparse graphs.

```
using vb = vec<bitset<200>>;
struct MaxClique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vec<Vertex> vv;
  vb e;
  vv V;
  vec<vi> C;
```
c51645,**c51645**
```
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
```
3d33ea,**89dd59**
```
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i])
T.push_back({v.i});
```
90a75f,**829efa**
```
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1,
1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
```
b9e912,**5e0488**
```
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
```
e33038,**06a583**
```
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  MaxClique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
```
69d5cc,**bb0ad8**
```
};
```
2145c1,**7d88d2**

## Edge Coloring

**description:** Given a simple, undirected graph with max degree $D$, computes a $(D + 1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
**time:** $O(NM)$

```
vi edgeColoring(int N, vec<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vec<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
```
546af3,**546af3**
```
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
```
2a0967,**3fa22c**
```
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
```

```
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
```
3a46bf,**30df56**
```
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```
6d5acf,**35919a**

## Directed MST

**description:** Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns $-1$.
**time:** $O(E \log V)$

```
struct Edge { int a, b; ll w; };
struct Node { /// lazy skew heap node
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
```
9353bd,**9353bd**
```
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
```
4d7158,**839210**
```
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }
pair<ll, vi> dmst(int n, int r, vec<Edge>& g) {
  RollbackUF uf(n);
  vec<Node*> heap(n);
```
bcaac1,**8acaf5**
```
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vec<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vec<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
```
21f6dc,**7dbada**
```
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) { /// found cycle, contract
        Node* cyc = 0;
```
b19d17,**cd204f**
```
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
```
57e33d,**200181**
```
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
```
eacfda,**9e1aeb**
```
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```
9d39bd,**7ed40b**

## Dominator Tree

**description:** Given a digraph, return the edges of the dominator tree given as an adjacency list (directed tree downwards from the root).
**time:** $O((V+E)\log V)$

```
vec<vi> dominator_tree(const vec<vi>& adj, int root) {
  int n = sz(adj) + 1, co = 0;
  vec<vi> ans(n), radj(n), child(n), sdomChild(n);
  vi label(n), rlabel(n), sdom(n), dom(n), par(n), bes(n);
  auto get = [&](auto self, int x) -> int {
    if (par[x] != x) {
      int t = self(self, par[x]);
      par[x] = par[par[x]];
```
3bb126,**3bb126**
```
      if (sdom[t] < sdom[bes[x]]) bes[x] = t;
    }
    return bes[x];
  };
  auto dfs = [&](auto self, int x) -> void {
    label[x] = ++co, rlabel[co] = x;
    sdom[co] = par[co] = bes[co] = co;
    for (auto y : adj[x]) {
```
5a01e5,**b16d92**
```
      if (!label[y])
        self(self, y), child[label[x]].push_back(label[y]);
      radj[label[y]].push_back(label[x]);
    }
  };
  dfs(dfs, root);
  for (int i = co; i >= 1; --i) {
    for (auto j : radj[i])
```
697a61,**69b95f**
```
      sdom[i] = min(sdom[i], sdom[get(get, j)]);
    if (i > 1) sdomChild[sdom[i]].push_back(i);
    for (auto j : sdomChild[i]) {
      int k = get(get, j);
      if (sdom[j] == sdom[k]) dom[j] = sdom[j];
      else dom[j] = k; }
    for (auto j : child[i]) par[j] = i;
  }
```
ef4d59,**36ce7f**
```
  for (int i = 2; i < co + 1; ++i) {
    if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
    ans[rlabel[dom[i]]].push_back(rlabel[i]);
  }
  return ans;
}
```
537ff0,**fbf9bd**

## Matroid Intersection

**description:** Algorithms for finding maximum independent sets in the intersection of two matroids. `oracle(i)` is independent after adding $i$, `oracle(i, j)` … after swapping $i, j$.
**time:** $O(r^2 n)$ for unweighted. For weighted $O(r^3 n)$ with SPFA $O(r^2 n \log n)$ with Dijkstra with potentials

```
template<typename Matroid1, typename Matroid2>
vi matroidIntersection(int n, Matroid1 M1, Matroid2 M2) {
  vec<bool> b(n);
  vi I[2];
  bool converged = false;
  while (!converged) {
    I[0].clear(), I[1].clear();
    for (int u = 0; u < n; u++) I[b[u]].push_back(u);
```
813382,**813382**
```
    M1.build(I[1]), M2.build(I[1]);
    vec<bool> target(n), pushed(n);
    queue<int> q;
    for (int u : I[0]) {
      target[u] = M2.oracle(u);
      if (M1.oracle(u)) pushed[u] = true, q.push(u);
    }
```
3176c3,**61a236**
```
    vi p(n, -1);
    converged = true;
    while (q.size()) {
      int u = q.front(); q.pop();
      if (target[u]) {
        converged = false;
        for (int v = u; v != -1; v = p[v]) b[v] = !b[v];
        break;
```
efa1f9,**cae275**
```
      }
      for (int v : I[!b[u]]) if (!pushed[v]) {
        if ((b[u] and M1.oracle(u, v)) or (b[v] and
M2.oracle(v, u)))
          p[v] = u, pushed[v] = true, q.push(v);
      }
    }
  }
  return I[1];
```
f79302,**289017**
```
}
template<typename T, typename Matroid1, typename Matroid2>
vi weightedMatroidIntersection(int n, vec<T> w, Matroid1 M1,
Matroid2 M2) {
  vec<bool> b(n), target(n), is_inside(n);
  vi I[2], from(n);
  vec<pair<T, int>> d(n);
  auto check_edge = [&](int u, int v) {
    return (b[u] and M1.oracle(u, v)) or (b[v] and
M2.oracle(v, u));
```
807c2b,**8f3b0a**

```
  };
  while (true) {
    I[0].clear(), I[1].clear();
    for (int u = 0; u < n; u++) I[b[u]].push_back(u);
    // I[1] contem o conjunto de tamanho I[1].size() de menor
peso
    M1.build(I[1]), M2.build(I[1]);
    for (int u = 0; u < n; u++) {
      target[u] = false, is_inside[u] = false, from[u] = -1;
      d[u] = {numeric_limits<T>::max(), INF};
    }
    deque<T> q;
    sort(I[0].begin(), I[0].end(), [&](int i, int j){ return
w[i] < w[j]; });
    for (int u : I[0]) {
      target[u] = M2.oracle(u);
      if (M1.oracle(u)) {
        if (is_inside[u]) continue;
        d[u] = {w[u], 0};
        if (!q.empty() and d[u] > d[q.front()])
q.push_back(u);
        else q.push_front(u);
        is_inside[u] = true;
      }
    }
    while (q.size()) {
      int u = q.front(); q.pop_front();
      is_inside[u] = false;
      for (int v : I[!b[u]]) if (check_edge(u, v)) {
        pair<T, int> nd(d[u].first + w[v], d[u].second + 1);
        if (nd < d[v]) {
          from[v] = u, d[v] = nd;
          if (is_inside[v]) continue;
          if (q.size() and d[v] > d[q.front()])
q.push_back(v);
          else q.push_front(v);
          is_inside[v] = true;
        }
      }
    }
    pair<T, int> mini = pair(numeric_limits<T>::max(), INF);
    int targ = -1;
    for (int u : I[0]) if (target[u] and d[u] < mini)
      mini = d[u], targ = u;
    if (targ != -1) for (int u = targ; u != -1; u = from[u])
      b[u] = !b[u], w[u] *= -1;
    else break;
  }
  return I[1];
}
```

9b7e59,71e0b9 f59f5e,72ede1 cd4158,90e9ae efd6c5,c45955 351a20,9e9956

# Flow

## Hopcroft Karp

**description:** Fast bipartite matching algorithm. Graph $g$ should be a list of neighbors of the left partition, and $r$ should be a vector full of $-1$'s of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex $i$ on the right side, or $-1$ if it's not matched.

**usage:** `vi r(m, -1); hopcroftKarp(g, r);`

**time:** $O(\sqrt{V}E)$

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

4cd757,4cd757 6bb083,731cfb

## Dinic

**time:** $O(VE \log U)$ where $U = \max|\text{cap}|$. $O(E \min(E^{\frac{1}{2}}, V^{\frac{2}{3}}))$ if $U = 1$. $O(\sqrt{V}E)$ for bipartite matching.

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, ll(0)); }
  };
  vi lvl, ptr, q;
  vec<vec<Edge>> g;
  Dinic(int n) : lvl(n), ptr(n), q(n), g(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    g[a].push_back({b, sz(g[b]), c, c});
    g[b].push_back({a, sz(g[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(g[v]); i++) {
      Edge& e = g[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, g[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random
data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : g[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

e5f9d7,e5f9d7 8a7457,2a7a44 36a59a,919e89 4aa386,9a5f07 43bffa,47a8a3 b902a8,ec708b

## Push Relabel

**description:** Push-relabel using the highest label selection rule and the gap heuristic.

**time:** $O(V^2\sqrt{E})$

```
struct PushRelabel {
  struct Edge {
    int to, rev;
    ll f, c;
    ll flow() { return max(f, ll(0)); }
  };
  vec<vec<Edge>> g;
  vec<ll> ec;
  vec<Edge*> cur;
  vec<vi> hs; vi H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
  }
  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.to][e.rev];
    if (!ec[e.to] && f) hs[H[e.to]].push_back(e.to);
    e.f += f; e.c -= f; ec[e.to] += f;
    back.f -= f; back.c += f; ec[back.to] -= f;
  }
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);
    for (int hi = 0;;) {
```

dc7894,bc6c17 3606ea,f74465

```
    while (hs[hi].empty()) if (!hi--) return -ec[s];
    int u = hs[hi].back(); hs[hi].pop_back();
    while (ec[u] > 0)  // discharge u
      if (cur[u] == g[u].data() + sz(g[u])) {
        H[u] = 1e9;
        for (Edge& e : g[u]) if (e.c && H[u] > H[e.to]+1)
          H[u] = H[e.to]+1, cur[u] = &e;
        if (++co[H[u]], !--co[hi] && hi < v)
          rep(i,0,v) if (hi < H[i] && H[i] < v)
            --co[H[i]], H[i] = v + 1;
        hi = H[u];
      } else if (cur[u]->c && H[u] == H[cur[u]->to]+1)
        addFlow(*cur[u], min(ec[u], cur[u]->c));
      else ++cur[u];
  }
 }
 bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```
88f514,54e987 02276d,a35a10 251436,84a67f

## Flow with Demands

**description:** To find minimal flow that satisifes demands you can binary search over the capacity on the $t - s$ edge.

```
template <class T> struct FlowDemands {
 int n;
 T mf;
 vec<ll> in, out;
 FlowDemands(int n): n(n), mf(n+2), in(n), out(n) {}
 void addEdge(int u, int v, ll cap, ll dem = 0) {
  mf.addEdge(u, v, cap - dem);
  out[u] += dem, in[v] += dem;
 }
 ll calc(int s, int t) {
  const ll inf = 1e9; // might need to be changed
  mf.addEdge(t, s, inf);
  rep(i,0,n) {
   mf.addEdge(n, i, in[i]);
   mf.addEdge(i, n+1, out[i]);
  }
  return mf.calc(n, n+1);
 }
};
```
61d6af,61d6af b45302,f9a644 475613,0fdd2a

## Min Cost Max Flow

**description:** To obtain the actual flow, look at positive values only.
**time:** $O(FE \log V)$ where F is max flow. $O(VE)$ for setpi.

```
#include <bits/extc++.h>
const ll INF = numeric_limits<ll>::max() / 4;
struct MCMF {
 struct Edge {
  int from, to, rev;
  ll cap, cost, flow;
 };
 int N;
 vec<vec<Edge>> ed;
 vi seen;
 vec<ll> dist, pi;
 vec<Edge*> par;
 MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N)
{}
 void addEdge(int from, int to, ll cap, ll cost) {
  if (from == to) return;
  ed[from].push_back({from,to,sz(ed[to]),cap,cost,0});
  ed[to].push_back({to,from,sz(ed[from])-1,0,-cost,0});
 }
 void path(int s) {
  fill(all(seen), 0);
  fill(all(dist), INF);
  dist[s] = 0; ll di;
  __gnu_pbds::priority_queue<pair<ll, int>> q;
  vec<decltype(q)::point_iterator> its(N);
  q.push({ 0, s });
  while (!q.empty()) {
   s = q.top().second; q.pop();
   seen[s] = 1; di = dist[s] + pi[s];
   for (Edge& e : ed[s]) if (!seen[e.to]) {
    ll val = di - pi[e.to] + e.cost;
    if (e.cap - e.flow > 0 && val < dist[e.to]) {
     dist[e.to] = val;
     par[e.to] = &e;
     if (its[e.to] == q.end())
      its[e.to] = q.push({ -dist[e.to], e.to });
     else
      q.modify(its[e.to], { -dist[e.to], e.to });
    }
   }
  }
  rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
 }
 pair<ll, ll> maxflow(int s, int t) {
  ll totflow = 0, totcost = 0;
  while (path(s), seen[t]) {
   ll fl = INF;
   for (Edge* x = par[t]; x; x = par[x->from])
    fl = min(fl, x->cap - x->flow);
   totflow += fl;
   for (Edge* x = par[t]; x; x = par[x->from]) {
    x->flow += fl;
    ed[x->to][x->rev].flow -= fl;
   }
  }
  rep(i,0,N) for(Edge& e : ed[i]) totcost += e.cost *
e.flow;
  return {totflow, totcost/2};
 }
 // if some costs can be negative, call this before:
 void setpi(int s) { // (otherwise, leave this out)
  fill(all(pi), INF); pi[s] = 0;
  int it = N, ch = 1; ll v;
  while (ch-- && it--)
   rep(i,0,N) if (pi[i] != INF)
    for (Edge& e : ed[i]) if (e.cap)
     if ((v = pi[i] + e.cost) < pi[e.to])
      pi[e.to] = v, ch = 1;
  assert(it >= 0); // negative cost cycle
 }
};
```
0346bf,80b4c0 2ec11c,7afcd2 3efd70,e84c66 143d8e,37b8dd 3bbfdd,f9e055 ab3667,7efdfd e96d2f,b8c44e 800bdb,13ff09

## Weighted Matching

**description:** Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes `cost[N][M]`, where `cost[i][j]` = cost for `L[i]` to be matched with `R[j]` and returns `(min cost, match)`, where `L[i]` is matched with `R[match[i]]`. Negate costs for max cost. Requires $N < M$.
**time:** $O(N^2M)$

```
pair<ll, vi> hungarian(const vec<vec<ll>> &a) {
 if (a.empty()) return {0, {}};
 int n = sz(a) + 1, m = sz(a[0]) + 1;
 vec<ll> u(n), v(m); vi p(m), ans(n - 1);
 rep(i,1,n) {
  p[0] = i;
  int j0 = 0; // add "dummy" worker 0
  vec<ll> dist(m, LLONG_MAX); vi pre(m, -1);
  vec<bool> done(m + 1);
  do { // dijkstra
   done[j0] = true;
   int i0 = p[j0], j1;
   ll delta = LLONG_MAX;
   rep(j,1,m) if (!done[j]) {
    auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
    if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
    if (dist[j] < delta) delta = dist[j], j1 = j;
   }
   rep(j,0,m) {
    if (done[j]) u[p[j]] += delta, v[j] -= delta;
    else dist[j] -= delta;
   }
   j0 = j1;
  } while (p[j0]);
  while (j0) { // update alternating path
   int j1 = pre[j0];
   p[j0] = p[j1], j0 = j1;
  }
 }
 rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
 return {-v[0], ans}; // min cost
}
```
a9b122,a9b122 c37721,21ddd9 d3f8b4,2631ce e82819,1c1eb6

## Blossom

**description:** Blossom algorithm for matching in general graphs.
**time:** $O(NM)$ though Benq disagrees...

```
vi blossom(vec<vi>& adj) {
 int n = adj.size(), T = -1;
 vi mate(n, -1), label(n), par(n), orig(n), aux(n, -1), q;
 auto lca = [&](int x, int y) {
  for (T++; ; swap(x, y)) {
   if (x == -1) continue;
   if (aux[x] == T) return x;
```

```
      aux[x] = T;
      x = (mate[x] == -1 ? -1 : orig[par[mate[x]]]);
    }
  };
  auto blossom = [&](int v, int w, int a) {
    while (orig[v] != a) {
      par[v] = w;
      w = mate[v];
      if (label[w] == 1) label[w] = 0, q.push_back(w);
      orig[v] = orig[w] = a, v = par[w];
    }
  };
  auto augment = [&](int v) {
    while (v != -1) {
      int pv = par[v], nv = mate[pv];
      mate[v] = pv, mate[pv] = v, v = nv;
    }
  };
  auto bfs = [&](int root) {
    fill(all(label), -1), iota(all(orig), 0);
    q.clear(), q.push_back(root), label[root] = 0;
    for (int i = 0; i < sz(q); i++) {
      int v = q[i];
      for (auto x : adj[v]) if (label[x] == -1) {
        label[x] = 1, par[x] = v;
        if (mate[x] == -1) return augment(x);
        label[mate[x]] = 0, q.push_back(mate[x]);
      } else if (label[x] == 0 && orig[v] != orig[x]) {
        int a = lca(orig[v], orig[x]);
        blossom(x, v, a), blossom(v, x, a);
      }
    }
  };
  // time halves if you start with (any) maximal matching.
  for (int i = 0; i < n; i++) if (mate[i] == -1) bfs(i);
  return mate;
}
```
676b47,676b47 4ccb85,d5f6e3 4e6fd9,cff070 356c23,18446f d57af5,92381b 7cf59c,7b4769

# Various

## Ternary Search

**description:** Find the smallest $i \in [a, b]$ that maximizes $f(i)$, assuming that $f(a) < ... < f(i) \geq ... \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $\leq$, and reverse the loop at (B).

**time:** $O(\log(b-a))$

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```
c62e94,c62e94

## Interval Container

**description:** Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive)

**time:** $O(\log n)$

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}
void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```
6d817b,6d817b 05ea01,969cd4 9dbda9,edce47

## Constant Intervals

**description:** Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback `g` for each such interval.

**usage:** `constantIntervals(0, sz(v), [&](int x) {return v[x];}, [&](int lo, int hi, T val) {...});`

**time:** $O\left(k \log \frac{n}{k}\right)$

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to-1);
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```
e7df27,e7df27 f0fdd4,e84301 b5fc67,753a4c

## Interval Cover

**description:** Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change while to add `|| R.empty()`. Returns empty set on failure (or if G is empty).

**time:** $O(n \log n)$

```
template<class T>
vi cover(pair<T, T> G, vec<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) {
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```
4937c7,4937c7 41486c,c980b5 f97f88,75d8bd

## Fast Knapsack

**description:** Given $n$ non-negative integer weights $w$ and a non-negative target $t$, computes the maximum $S \leq t$ such that $S$ is the sum of some subset of the weights.

**time:** $O(n \cdot \max w_i)$

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```
682d61,682d61 83ac30,b20ccc

## Divide and Conquer DP

**description:** Given $a_i = \min_{\text{lo}(i) \le k < \text{hi}(i)} f(i,k)$ where the (minimal) optimal $k$ increases with $i$, computes $a_i$ for $i \in [L, R)$.
**time:** $O((N + (\text{hi} - \text{lo})) \log N)$

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;                          13ddb0,13ddb0
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
                                                     8cf7aa,97a9dd
};                                                   2145c1,d38d2b
```

## Knuth DP Optimization

**description:** When doing DP on intervals: $a_{i,j} = \min_{i<k<j} a_{i,k} + a_{k,j} + f(i,j)$, where the (minimal) optimal $k$ increases with both $i$ and $j$, one can solve intervals in increasing order of length, and search $k = p_{i,j}$ for $a_{i,j}$ only between $p_{i,j-1}$ and $p_{i+1,j}$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$.
**time:** $O(N^2)$

## Combinatorial

### Permutations

#### Cycles

Let $g_S(n)$ be the number of $n$-permutations whose cycle lengths all belong to the set $S$. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

#### Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

#### Burnside's lemma

Given a group $G$ of symmetries and a set $X$, the number of elements of $X$ *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g \in G} |X^g|,$$

where $X^g$ are the elements fixed by $g$ ($g.x = x$).
If $f(n)$ counts configurations' (of some sort) of length $n$, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k \mid n} f(k)\phi(n/k).$$

### Partitions and subsets

#### Partition function

Number of ways of writing $n$ as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

#### Lucas' Theorem

Let $n, m$ be non-negative integers and $p$ a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

### General purpose numbers

#### Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able). $B[0, ...] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, ...]$
Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m} \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$
$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

#### Stirling numbers of the first kind

Number of permutations on $n$ items with $k$ cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)...(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, ...$

#### Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly $k$ elements are greater than the previous element. $k$ $j$:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ $j$:s s.t. $\pi(j) \ge j$, $k$ $j$:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

#### Stirling numbers of the second kind

Partitions of $n$ distinct elements into exactly $k$ groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

#### Bell numbers

Total number of partitions of $n$ distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, ....$ For $p$ prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

#### Labeled unrooted trees

hash on $n$ vertices: $n^{n-2}$
hash on $k$ existing trees of size $n_i$: $n_1 n_2 \cdots n_k n^{k-2}$
hash with degrees $d_i$: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

#### Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, ...$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with $n$ pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

## Number Theory

**Pythagorean Triples** The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ b = k \cdot (2mn),\ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either $m$ or $n$ even.

**Primes** $p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1thin 000thin 000.
Primitive roots exist modulo any prime power $p^a$, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^{\times}$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

**Estimates** $\sum_{d \mid n} d = O(n \log \log n)$.
The number of divisors of $n$ is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200000 for $n < 1e19$.

### Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d \mid n} f(d) \Leftrightarrow f(n) = \sum_{d \mid n} \mu(d) g(n/d)$$

Other useful formulas/forms:
$\sum_{d \mid n} \mu(d) = [n = 1]$ (very useful)
$g(n) = \sum_{n \mid d} f(d) \Leftrightarrow f(n) = \sum_{n \mid d} \mu(d/n) g(d)$
$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$